



Problemi QUBO e di Feature Reduction

Michele Marchesi

AA 2025-26

Premessa: notazione matriciale

- La trattazione seguente presuppone le seguenti nozioni sulle matrici:
 - concetto di matrice e di vettore, dimensioni, vettore colonna e riga
 - notazione per matrici e loro elementi, trasposizione di matrici e vettori
 - prodotto di uno scalare per un vettore e per una matrice
 - prodotto scalare di due vettori
 - somma di matrici, prodotto di matrici, prodotto di matrice per vettore colonna e di vettore riga per matrice
 - diagonale di una matrice, operatore $\text{diag}(\mathbf{x})$
 - matrici quadrate: diagonali, simmetriche, triangolari (upper e lower)
 - forma quadratica ($\mathbf{x}_{n \times 1}$, $A_{n \times n}$) :
$$\mathbf{x}^T A \mathbf{x} = \sum_{i=1}^n \sum_{j=1}^n a_{ij} x_i x_j$$

Problemi QUBO

- QUBO: ottimizzazione binaria quadratica non vincolata – Quadratic Unconstrained Binary Optimization
- QUBO significa trovare il minimo o il massimo) di una funzione quadratica, qui espressa in forma matriciale: $f(\mathbf{x}) = a + \mathbf{b}^T \mathbf{x} + \mathbf{x}^T [\mathbf{Q}] \mathbf{x}$, dove:
 - $\mathbf{x} = [x_1, x_2, \dots, x_n]^T$ è un vettore di componenti binarie
 - $x_i \in \{0, 1\}$, oppure: $x_i \in \{-1, 1\}$, $i = 1, \dots, n$
 - a e le componenti di $\mathbf{b}$ e $[\mathbf{Q}]$ sono numeri reali
 - Non ci sono vincoli sul dominio di $f(\mathbf{x})$
- In forma estesa: $f(\underline{x}) = a + \sum_i b_i x_i + \sum_i \sum_j q_{ij} x_i x_j$



Problemi che possono essere risolti usando QUBO

- Problemi di assegnamento quadratico
- Problemi di capital budgeting
- Problemi dello zaino multiplo
- Problemi di allocazione dei task (sistemi distribuiti)
- Problemi di massima diversità
- Problemi p-mediana
- Problemi di assegnamento asimmetrico
- Problemi di assegnamento simmetrico
- Problemi di assegnamento con vincoli aggiuntivi
- Problemi dello zaino quadratico
- Problemi di soddisfacimento di vincoli (CSP)
- Problemi di tomografia discreta
- Problemi di partizionamento di insiemi
- Problemi di set packing
- Problemi di localizzazione di magazzini
- Problemi di clique massima
- Problemi del commesso viaggiatore
- Problemi di taglio massimo
- Problemi di colorazione dei grafi
- Problemi di partizionamento di numeri
- Problemi di ordinamento lineare
- Problemi di partizionamento in clique
- Problemi SAT

Variabili QUBO

- Le variabili QUBO sono vettori di valori binari: $\mathbf{x}^T = [0, 1, 1, 1, 0, 0, 1, \dots]$
- Forma binaria o QUBO: $x_i \in \{0, 1\}, i = 1, \dots, n$
- Forma spin o Ising: $s_i \in \{-1, 1\}, i = 1, \dots, n$
- Conversioni: $x_i = (s_i + 1) / 2$; $s_i = 2x_i - 1$
- La forma binaria è più usata perché in forma binaria $x_i^2 = x_i$, ed è immediato anche rappresentare il complemento di una variabile:

$$x_i^2 = x_i \quad (\text{formule semplificate})$$

$$\bar{x}_i = 1 - x_i \quad (\bar{0} \equiv 1, \bar{1} \equiv 0)$$

Formulazione standard per variabili binarie

- Usiamo la minimizzazione (equivalente a massimizzare l'opposto)
- Quando possibile, usiamo la formulazione matriciale: $f(\underline{x}) = a + \underline{b}^T \underline{x} + \underline{x}^T Q \underline{x}$
- Nella minimizzazione, **il termine costante a può essere trascurato**
- Il termine lineare $\underline{b}^T \underline{x}$ può diventare “quadratico” perché $x_i^2 = x_i$:

$$\underline{b}^T \underline{x} = \sum_i b_i x_i = \sum_i b_i x_i^2 = \underline{x}^T \text{Diag}(\underline{b}) \underline{x}$$

- **Il problema diventa la minimizzazione di:** $f(\underline{x}) = \underline{x}^T Q' \underline{x}$
- dove: $Q' = Q + \text{Diag}(\underline{b})$

Ancora sulla matrice Q

- I termini fuori diagonale della matrice quadrata Q sono i coefficienti dei prodotti: $q_{ij}x_i x_j, q_{ji}x_j x_i$ $i \neq j$, presenti nella somma che rappresenta la forma quadratica:
$$\underline{x}^T Q \underline{x}$$

- Poiché $x_i x_j = x_j x_i$, ciò che conta davvero è la somma $q_{ij} + q_{ji}$

- Quindi, la matrice Q viene di solito sostituita da:

- una matrice simmetrica Q^s i cui termini sono: $q_{ij}^s = q_{ji}^s = \frac{q_{ij} + q_{ji}}{2}, i \neq j$

- oppure una matrice triangolare superiore Q^u i cui termini sono:

$$q_{ii}^u = q_{ii}; q_{ij}^u = q_{ij} + q_{ji}, j > i; q_{ij}^u = 0, j < i$$



Modelli QUBO

- I modelli QUBO appartengono a una classe di problemi noti come NP-hard.
- In pratica, i solutori esatti progettati per trovare soluzioni “ottime” hanno poche probabilità di successo, tranne che per istanze molto piccole.
- Problemi di dimensioni realistiche possono richiedere giorni o settimane senza produrre soluzioni di alta qualità.
- Fortunatamente, i moderni metodi metaeuristici stanno ottenendo risultati notevoli: sono progettati per trovare soluzioni di alta qualità, anche se non necessariamente ottime, in tempi di calcolo contenuti.

Un primo esempio

- Minimizzare: $y = -5x_1 - 3x_2 - 8x_3 - 6x_4 + 4x_1x_2 + 8x_1x_3 + 2x_2x_3 + 10x_3x_4$
dove le variabili x_i sono binarie.

- La funzione da minimizzare è una funzione quadratica in variabili binarie, con una parte lineare: $-5x_1 - 3x_2 - 8x_3 - 6x_4$ e una parte quadratica:

$$4x_1x_2 + 8x_1x_3 + 2x_2x_3 + 10x_3x_4$$

- Poiché le variabili binarie soddisfano $x_i^2 = x_i$, la parte lineare può essere scritta come:

$$-5x_1^2 - 3x_2^2 - 8x_3^2 - 6x_4^2$$

Un primo esempio

- Possiamo quindi riscrivere il modello nella seguente forma matriciale simmetrica:

$$\text{Minimize } y = (x_1 \ x_2 \ x_3 \ x_4) \begin{bmatrix} -5 & 2 & 4 & 0 \\ 2 & -3 & 1 & 0 \\ 4 & 1 & -8 & 5 \\ 0 & 0 & 5 & -6 \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \\ x_3 \\ x_4 \end{bmatrix} = \underline{\mathbf{x}}^T \mathbf{Q} \underline{\mathbf{x}}$$

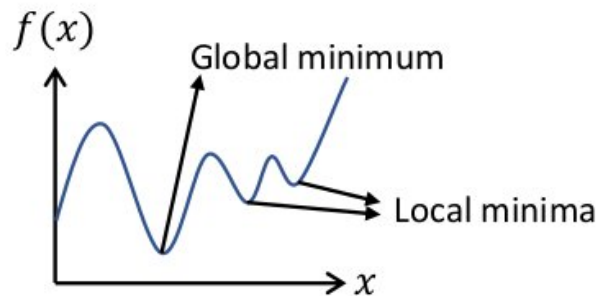
- Si noti che i coefficienti dei termini lineari originali compaiono sulla diagonale principale della matrice $\mathbf{Q}$.
- La soluzione del modello precedente è:
 $x_1 = x_4 = 1, x_2 = x_3 = 0$; ovvero: $\mathbf{x} = [1, 0, 0, 1] \rightarrow y = -11$

Creare modelli QUBO usando penalità note

- Molti problemi di interesse includono vincoli aggiuntivi da rispettare.
- Questi modelli vincolati possono essere riformulati come QUBO aggiungendo **penalità quadratiche** alla funzione obiettivo.
- Le penalità valgono zero per le soluzioni ammissibili e assumono un valore positivo per quelle non ammissibili.
- In minimizzazione si ottiene una funzione obiettivo aumentata: se le penalità sono nulle, resta la funzione originale.
- Nel caso della *feature reduction* non ci sono vincoli aggiuntivi, quindi non li tratteremo

Ottimizzazione

- L'ottimizzazione matematica è il processo di ricerca di x tale che $f(x)$ sia minimizzata, per una certa funzione $f: \mathbb{R}^n \rightarrow \mathbb{R}$ di interesse.
- L'ottimizzazione può essere vincolata da disuguaglianze e/o uguaglianze, ad esempio cercare x nell'intervallo $0 \leq x \leq 10$ tale che una certa $f(x)$ sia minimizzata.
- I problemi di ottimizzazione si dividono in tre grandi categorie:
 - Ottimizzazione risolvibile algebricamente, ad esempio trovare x che minimizza $f(x) = x^2 - 2x - 5$.
 - Ottimizzazione convessa, per funzioni con un unico minimo "globale" e per le quali esistono algoritmi efficienti.
 - Ottimizzazione generale, che riguarda funzioni qualsiasi, che possono quindi avere più minimi "locali":
è quella che ci interessa:



Metodi di ottimizzazione

- La maggior parte dei metodi generali per minimizzare $f(\mathbf{x})$ procede nel modo seguente:
 - 1 Definisce un punto iniziale $\mathbf{x}_0$
 - 2 Pone $\mathbf{x}_{\text{opt}} = \mathbf{x}_0$, $f_{\text{opt}} = f(\mathbf{x}_0)$
 - 3 Genera un punto di prova, $\mathbf{x}_t$, nel *vicinato* di $\mathbf{x}_{\text{opt}}$
 - 4 Se $f(\mathbf{x}_t) \leq f(\mathbf{x}_{\text{opt}})$, allora pone $\mathbf{x}_{\text{opt}} = \mathbf{x}_t$, $f_{\text{opt}} = f(\mathbf{x}_t)$
 - 5 Applica politiche specifiche se $f(\mathbf{x}_t) > f(\mathbf{x}_{\text{opt}})$, per evitare di restare intrappolati in minimi locali
 - 6 Se il criterio di terminazione non è soddisfatto, torna al passo 3
 - 7 La soluzione è: $\mathbf{x}_{\text{opt}}$



Tabu Search

- Tabu Search (TS) è una metaeuristica per l'ottimizzazione.
- Mantiene “liste tabu” di mosse temporaneamente vietate, per evitare cicli attorno a ottimi locali.
- Le liste hanno dimensione finita: dopo un certo tempo, le mosse vengono rimosse.
- La memoria della ricerca può essere organizzata su tre orizzonti:
 - breve: evita oscillazioni attorno a un minimo locale;
 - medio: orienta la ricerca verso aree promettenti;
 - lungo: spinge la ricerca verso nuove regioni quando essa resta bloccata.

Simulated Annealing

- Il simulated annealing (SA) è una metaeuristica ispirata al processo fisico di cristallizzazione con *ricottura*. Si applica a problemi di ottimizzazione combinatoria o continua in cui ci sia una nozione definita di vicinato.
- Scelto un $\mathbf{x}_0$ iniziale, l'algoritmo procede così:
 - 1 $\mathbf{x} = \mathbf{x}_0$, valuta $f(\mathbf{x})$; , $f_{\text{opt}} = f(\mathbf{x}) \leftarrow$ qui $\mathbf{x}$ è $\mathbf{x}_{\text{opt}}$
 - 2 Sceglie un *vicino* $\mathbf{x}'$ di $\mathbf{x}$ con qualche strategia, spesso con componenti random
 - 3 Valuta $f(\mathbf{x}')$
 - 4 Se $f(\mathbf{x}') \leq f(\mathbf{x})$, pone $\mathbf{x} \leftarrow \mathbf{x}'$, $f_{\text{opt}} = f(\mathbf{x}')$
Altrimenti, se $f(\mathbf{x}') \geq f(\mathbf{x})$, decide in modo random se porre:
 $\mathbf{x} \leftarrow \mathbf{x}'$, $f_{\text{opt}} = f(\mathbf{x}')$ oppure mantenere $\mathbf{x}$ invariato
 - 5 Ripete per un numero prefissato di iterazioni
 - 6 All fine, $\mathbf{x}_{\text{opt}}$ è posto al valore di $\mathbf{x}$ per cui la funzione trovata era minima

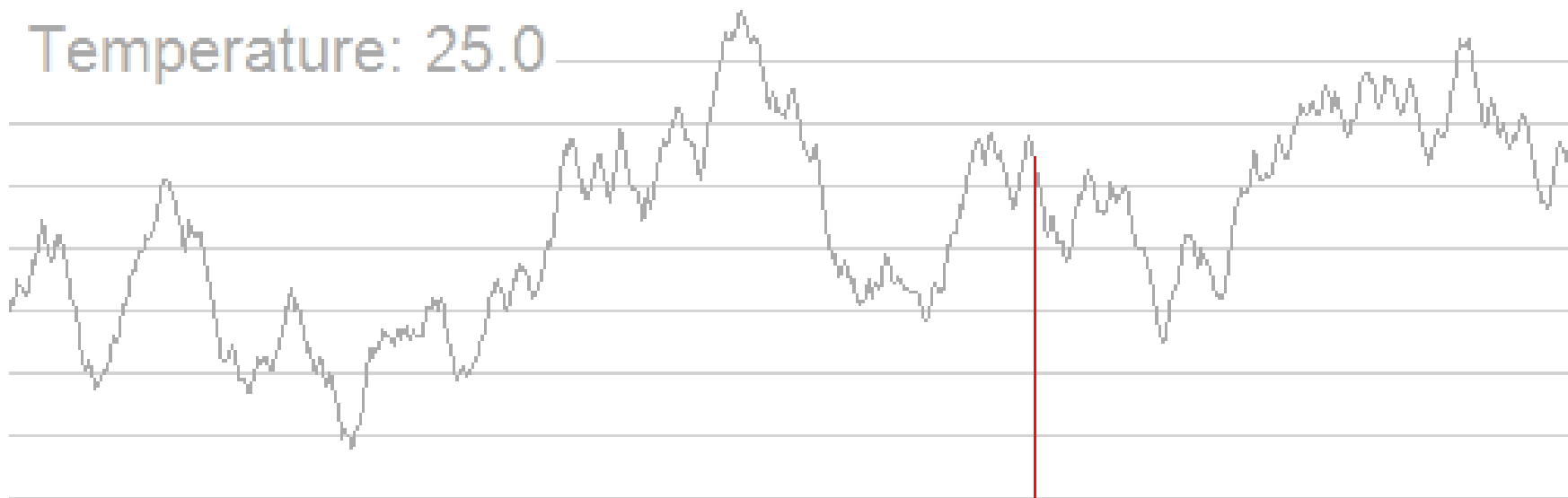
Simulated Annealing

- Naturalmente, la decisione casuale del passo 4 deve essere tale che, quanto più $f(\mathbf{x}')$ supera $f(\mathbf{x})$, tanto meno probabile è accettare $\mathbf{x} \leftarrow \mathbf{x}'$.
- Inoltre, questo processo casuale cambia durante le iterazioni: le mosse “in salita” sono **più probabili all’inizio** dell’ottimizzazione e **meno probabili verso la fine**.
- In questo modo, all’inizio domina l’**esplorazione** dell’intero dominio di f , mentre alla fine domina il **raffinamento** della soluzione trovata.
- Ispirandosi al processo fisico di ricottura, la probabilità di accettazione è solitamente determinata da:

$$p(\text{accept}) = \exp \left(-\frac{f(x') - f(x)}{T} \right)$$

dove T è la “temperatura”, che viene “raffreddata” durante l’esecuzione, rendendo meno probabili le mosse in salita.

Ricerca del massimo con Simulated Annealing



(https://en.wikipedia.org/wiki/File:Hill_Climbing_with_Simulated_Annealing.gif)

Simulated annealing: un ricordo personale

- Un articolo pubblicato da me e dai miei coautori nel 1987!
- Questo algoritmo riguarda la minimizzazione globale di funzioni continue
- È ancora piuttosto usato:
 - oltre 2200 citazioni su Google Scholar (1328 su Scopus)
 - ancora circa 30-40 citazioni all'anno
 - uno degli articoli più citati mai pubblicati su ACM TOMS

Minimizing Multimodal Functions of Continuous Variables with the “Simulated Annealing” Algorithm

A. CORANA, M. MARCHESI, C. MARTINI, and S. RIDELLA
Istituto per i Circuiti Elettronici-C.N.R.

A new global optimization algorithm for functions of continuous variables is presented, derived from the “Simulated Annealing” algorithm recently introduced in combinatorial optimization.

The algorithm is essentially an iterative random search procedure with adaptive moves along the coordinate directions. It permits uphill moves under the control of a probabilistic criterion, thus tending to avoid the first local minima encountered.

The algorithm has been tested against the Nelder and Mead simplex method and against a version of Adaptive Random Search. The test functions were Rosenbrock valleys and multimodal functions in 2, 4, and 10 dimensions.

The new method proved to be more reliable than the others, being always able to find the optimum, or at least a point very close to it. It is quite costly in terms of function evaluations, but its cost can be predicted in advance, depending only slightly on the starting point.

Categories and subject descriptors: G.1.6. [Numerical Analysis]: Optimization—*nonlinear programming*; G.3 [Mathematics of Computing]: Probability and Statistics—*probabilistic algorithms (including Monte Carlo)*; G.4. [Mathematics of Computing]: Mathematical Software—*certification and testing*

General Terms: Algorithms, Performance

Additional Key Words and Phrases: Global optimization, stochastic optimization, test functions



Software metaeuristico per QUBO

- La maggior parte delle implementazioni open source di solutori QUBO generali è fornita da D-Wave, da usare insieme ai loro computer di quantum annealing
- La maggior parte di questi solutori è scritta in C o C++ e dispone di un'interfaccia Python
- Un altro fornitore di software open source è Jij (<https://www.j-ij.com/>) con OpenJij, focalizzato su SA e con integrazione D-Wave. Si veda anche: <https://github.com/Jij-Inc/OpenJij>
- Lo strumento qubolite è fornito dal Lamarr Institute for Machine Learning and Artificial Intelligence: <https://lamarr-institute.org/blog/qubolite-qubo/>



Sampler e solutori D-Wave

- **Random:** un sampler che genera campioni casuali uniformi
- **Simulated Annealing:** un'euristica probabilistica per l'ottimizzazione e il campionamento di Boltzmann approssimato, adatta a trovare buone soluzioni per problemi grandi.
- **Steepest Descent:** analogo discreto della discesa del gradiente, spesso usato nel machine learning, che trova rapidamente un minimo locale.
- **Tabu:** un'euristica che usa ricerca locale con metodi per uscire dai minimi locali.

Sampler e solutori D-Wave

- Il solutore Tabu Search, scritto in C++, si trova qui:
<https://github.com/dwavesystems/dwave-tabu>
 - Implementa l'algoritmo proposto da G. Palubeckis, "Multistart tabu search strategies for the unconstrained binary quadratic optimization problem." Annals of Operations Research 131 (2004).
- Il solutore Simulated Annealing si trova qui:
<https://github.com/dwavesystems/dwave-neal>
- L'elenco completo dei sampler D-Wave è disponibile qui:
<https://github.com/dwavesystems/dwave-samplers>



Feature Reduction con QUBO



Feature Reduction con QUBO

- Problema: classificare i campioni di un dataset
 - dataset: un insieme di campioni
 - campione: un record con vari campi (*features*), da classificare in due classi (vero/falso, sano/malato, normale/anomalo,...)
- Si usa un *classificatore*, cioè un algoritmo che:
 - *Apprende* avendo in input un dataset simile al precedente, coi campioni già correttamente classificati (learning set)
 - Poi, dato un campione del dataset da classificare, valuta a quale classe questo appartiene

Perché ridurre le feature?

- Un dataset con molte feature non è sempre migliore: può essere ridondante, rumoroso o costoso da gestire.
- Meno feature significa modelli più semplici, più interpretabili e spesso più robusti.
- Feature molto correlate tra loro possono portare informazione duplicata.
- Feature poco legate al target possono comportarsi come rumore.
- Lo scopo della selezione è: ***trovare poche feature “indipendenti” e “influenti”***

Un semplice esempio

- Immaginiamo di voler classificare clienti in due classi: 0 = basso rischio, 1 = alto rischio. Il dataset è:

reddito	debito	età	saldo	debito/ reddito	target
2400	900	31	1200	0.38	0
1800	1600	48	-200	0.89	1
3200	700	28	2100	0.22	0
1500	1900	52	-500	1.27	1

- Alcune colonne sono informative, altre duplicate o poco utili:
 - “debito/reddito” è molto informativa ma deriva da due feature già presenti.
 - “età” può essere meno correlata al target, a seconda del dataset.
 - QUBO sceglie la combinazione ottima: $\mathbf{x} = [1, 0, 0, 1, 1]$, ove:
 - 1 = la feature è usata dal classificatore; 0 = la feature viene scartata

Elaborazioni complessive

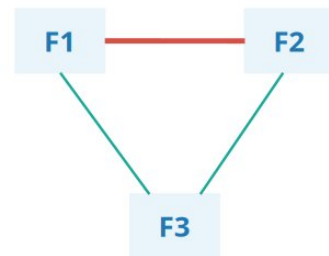
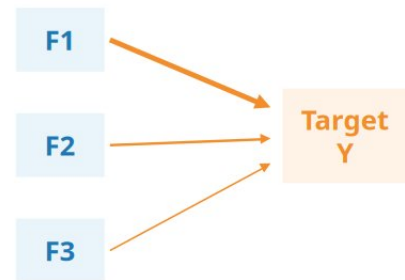
- La feature reduction è una fase intermedia tra preparazione dati e classificazione:



- Output finale: predizione 0/1 e, idealmente, uno score/probabilità della classe positiva.

Due criteri: influenza e indipendenza

- Le feature migliori sono:
- Influenti: correlate col valore target Y:**
Alta correlazione con Y $\Rightarrow$ feature rilevante
- Indipendenti: poco correlate** con altre feature:
Evitare coppie di feature fortemente correlate



Variabili binarie di selezione

- Ogni feature è associata a una variabile binaria: scelta sì/no:

F1	F2	F3	F4	F5	F6	F7	F8
1	0	1	0	0	1	0	0

- $x_j \in \{0, 1\}$; se $x_j = 1$ la feature è usata, se $x_j = 0$ la feature è esclusa
- La soluzione del problema QUBO è un vettore $\mathbf{x}^*$ che identifica il sottoinsieme scelto.
- Con m feature, ci sono 2^m possibili soluzioni: serve un criterio di ottimizzazione.

Formalizzazione del problema QUBO

- Il dataset può essere rappresentato come una matrice di features $[U]$, le cui righe corrispondono agli m campioni e le cui colonne alle n features:
- I valori target sono contenuti in un vettore $\mathbf{v}$ con m elementi (nelle formule denotato come “ V ”), che corrispondono alle righe di medesimo indice:
- Gli elementi di $\mathbf{v}$, v_i , possono assumere solo i valori binari 0 (credito in bonis) o 1 (credito in sofferenza).

$$U = \begin{bmatrix} u_{11} & u_{12} & u_{13} & \dots & u_{1n} \\ u_{21} & u_{22} & u_{23} & \dots & u_{2n} \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ u_{m1} & u_{m2} & u_{m3} & \dots & u_{mn} \end{bmatrix}$$

$$V = \begin{bmatrix} v_1 \\ v_2 \\ \vdots \\ v_m \end{bmatrix}$$

Formalizzazione del problema QUBO (2)

- Dall'insieme originale di n features vogliamo selezionare un sottoinsieme di K features.
- Il nostro obiettivo è trovare le colonne di U che sono **correlate con V** , ma **non correlate tra loro**.
- Supponiamo che il valore dei coefficienti di correlazione possa assumere valori compresi tra -1 e 1.
- Sia ρ_{ij} la correlazione tra la colonna i e la colonna j della matrice U , $i \neq j$, e sia ρ_{vj} la correlazione tra la colonna j di U e v .
- Introduciamo poi n variabili binarie x_j , che hanno la proprietà: $x_j = 1$, se la feature j è selezionata, $x_j = 0$ altrimenti.

$$X = \begin{bmatrix} x_1 \\ x_2 \\ \vdots \\ x_m \end{bmatrix}$$

Formalizzazione del problema QUBO (3)

- Il sottoinsieme migliore di feature è dato dal valore di $\mathbf{x}$ che minimizza una funzione QUBO, che costruiamo a partire da due componenti.
- La prima componente rappresenta l'**influenza** che le feature hanno sulla classe da determinare; essa aumenta all'aumentare del numero di feature selezionate:
- La seconda componente rappresenta l'**indipendenza** delle feature. L'espressione mostrata aumenta all'aumentare del numero di feature entrambe selezionate e correlate tra loro, che è l'opposto dell'indipendenza:
- Si noti che si usa sempre il modulo delle correlazioni!

$$\sum_{j=1}^n x_j |\rho_{V_j}|.$$

$$\sum_{j=1}^n \sum_{\substack{k=1, \\ k \neq j}}^n x_j x_k |\rho_{jk}|$$

Formalizzazione del problema QUBO (4)

- Per ottenere una funzione obiettivo che sia massima nel punto ottimale, dovremo sottrarre la seconda formula dal primo termine.
- Combiniamo i due termini utilizzando un parametro α ($0 \leq \alpha \leq 1$) per rappresentare il peso che vogliamo assegnare all'indipendenza (massima per $\alpha = 0$) e all'influenza (massima per $\alpha = 1$).
- Quindi neghiamo l'espressione complessiva per ottimizzare al minimo, ottenendo:

$$f(\mathbf{x}) = - \left[\alpha \sum_{j=1}^n x_j |\rho_{Vj}| - (1 - \alpha) \sum_{j=1}^n \sum_{\substack{k=1, \\ k \neq j}}^n x_j x_k |\rho_{jk}| \right].$$

Formalizzazione del problema QUBO (5)

$$f(\mathbf{x}) = - \left[\alpha \sum_{j=1}^n x_j |\rho_{Vj}| - (1 - \alpha) \sum_{j=1}^n \sum_{\substack{k=1, \\ k \neq j}}^n x_j x_k |\rho_{jk}| \right].$$

- Il termine a sinistra di primo grado si può incorporare nella 2^a sommatoria sfruttando la proprietà che $x_j = x_j x_j$ per le variabili binarie
- Possiamo riscrivere la sommatoria come prodotto vettoriale: $f(\mathbf{x}) = -\mathbf{x}^T Q \mathbf{x}$.
- Ove: $q_{jk} = (1 - \alpha) |\rho_{jk}|$, $j \neq k$; $q_{jj} = -\alpha |\rho_{Vj}|$
- Da questa formula, possiamo esprimere il problema in termini dell'operatore *argmin*, che restituisce il vettore $\mathbf{x}^*$ per il quale il suo argomento (una funzione) è minimizzato:
- Si tratta del classico problema QUBO!

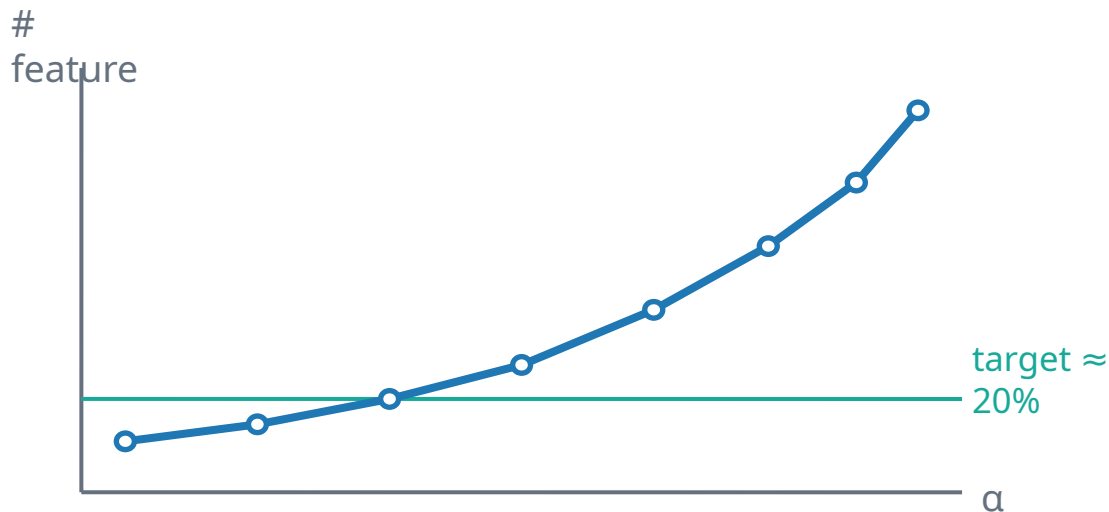
$$\mathbf{x}^* = \underset{\mathbf{x}}{\operatorname{argmin}} \left[-\mathbf{x}^T Q \mathbf{x} \right].$$

Formalizzazione del problema QUBO (6)

- Le correlazioni tra le colonne di U (le singole features per tutti i campioni) e il vettore target $\mathbf{v}$, ρ_{vj} , e tra le coppie di colonne, ρ_{ij} , è calcolata usando la formula di Spearman.
- Una volta trovato il vettore $\mathbf{x}$ con l'indicazione delle feature ottime, la matrice U si riduce eliminandone le colonne che corrispondono ai valori $x_j = 0$, e si utilizza la matrice U' ridotta per la classificazione.
- Il risultato dell'ottimizzazione QUBO dipende da α . Per piccoli valori di α si troveranno meno colonne di U , mentre per $\alpha = 1$ l'ottimizzazione restituirà necessariamente tutte le colonne ($U' = U$).
- È quindi necessario effettuare varie prove al variare di α , per determinare il numero ottimale di features da eliminare, oltre che quali eliminare.

Il parametro α controlla il compromesso

- Nel nostro caso vogliamo mantenere il 20% delle feature
- Variando α cambia il numero e il tipo di feature selezionate:



Procedura pratica:

1. provare più valori di α , in sequenza
2. risolvere il problema QUBO
3. scegliere l' α che produce il 20% di feature
4. il classificatore userà le feature indicate in $\mathbf{x}^*$

Risoluzione del problema QUBO

- Assumiamo di avere una libreria o algoritmo capace di minimizzare il problema QUBO:

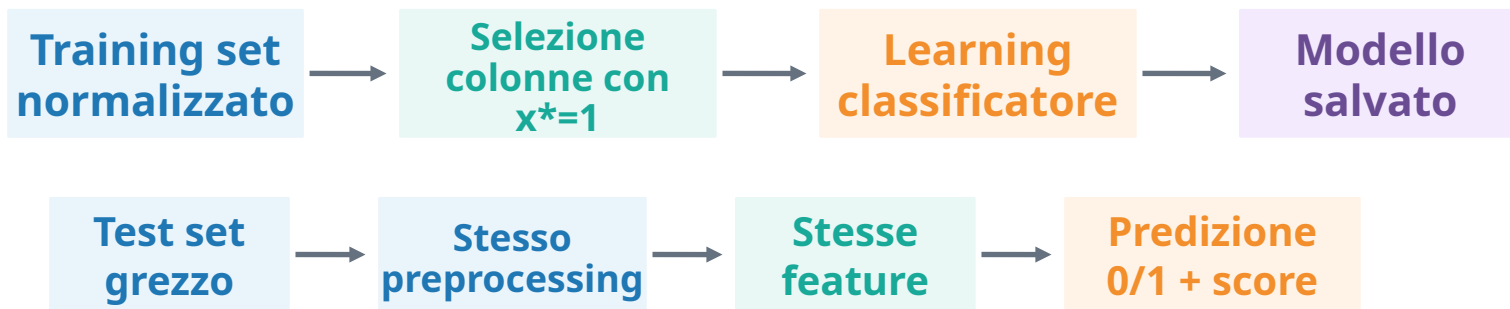


$$\mathbf{x}^* = [1, 0, 0, 1, 1, 0, \dots]$$

- $\mathbf{x}^*$ L'output viene convertito nella lista delle colonne delle feature selezionate.
- Si crea un nuovo dataset eliminando le colonne non selezionate
- La qualità della soluzione si valuta sul classificatore, non sul valore della funzione costo QUBO

Dalle feature selezionate al classificatore

- Il vettore x^* definisce quali colonne entrano nella fase di learning.



- I parametri del classificatore si “imparano” sul training set e si riusano sul test set.
- I dati di rischio di credito sono molto “sbilanciati” (i clienti a rischio sono tra l’1% e il 2% del totale): questo complica la classificazione

Conclusioni

- La selezione QUBO collega preprocessing, ottimizzazione e classificazione in una sequenza di elaborazione unica.
- Punti chiave:
 - 1) Il problema non è solo “trovare le feature più correlate”, ma scegliere un sottoinsieme coerente e poco ridondante.
 - 2) L'approccio QUBO usa variabili binarie per rappresentare le decisioni di selezione delle colonne.
 - 3) Il parametro α permette di regolare il compromesso tra indipendenza e influenza.
 - 4) La bontà finale si misura sul classificatore: meno feature, prestazioni simili o migliori, maggiore interpretabilità.

Riferimenti suggeriti

- F. Glover, G. Kochenberger, Yu Du, “Quantum Bridge Analytics I: A Tutorial on Formulating and Using QUBO Models”, Ann Oper Res 314, 141–183 (2022). <https://link.springer.com/article/10.1007/s10479-022-04634-2>
- Andrew Milne, Maxwell Rounds, and Phil Goddard, “Optimal feature selection in credit scoring and classification using a quantum annealer”, Technical report, 1QB Information Technologies, 2017. <https://api.semanticscholar.org/CorpusID:203690617>. (utile per la descrizione del problema, non per l’algoritmo specifico usato).